

Contents

Introduction à Python	1
1 Qu'est-ce que Python ?	1
2 Types de projets avec Python	2
3 Installation de Python	2
Sur Windows	3
Sur Linux (Ubuntu/Debian)	3
Sur macOS	3
4 Premier script Python	3
5 Structuration d'un projet Python	4
6 Gestion des dépendances	4
7 Environnements virtuels	5
8 Outils avancés pour l'environnement	5
9 Bonnes pratiques dès le début	5
Résumé	6

Introduction à Python

Le support est une propriété de **Bechir Bejaoui**

1 Qu'est-ce que Python ?

Python est un langage de programmation de haut niveau, créé par Guido van Rossum en 1991.

Il est connu pour sa syntaxe simple et lisible, inspirée de l'anglais.

Caractéristiques clés :

- **Interprété** : Le code est exécuté ligne par ligne par un interpréteur (comme CPython), sans compilation préalable en code machine. Cela facilite le débogage mais peut être plus lent que les langages compilés comme C++.
- **Dynamiquement typé** : Les types de variables sont déterminés à l'exécution. Exemple : `x = 5` (int), puis `x = "texte"` (str) sans déclaration explicite. Cela rend le code flexible mais nécessite une vigilance pour éviter les erreurs de type.
- **Orienté objet (OOP)** : Supporte les classes, l'héritage, le polymorphisme, etc., mais permet aussi d'écrire du code procédural ou fonctionnel.
- **Fonctionnel** : Supporte les paradigmes fonctionnels (fonctions de première classe, lambdas, map/filter/reduce), mais n'est pas purement

fonctionnel comme Haskell.

- **Multi-paradigme** : Peut être utilisé de manière impérative, OOP, fonctionnelle ou procédurale.
- **Gestion automatique de la mémoire** : Garbage collector pour libérer la mémoire inutilisée.

Python est “batteries included” : bibliothèque standard riche (math, datetime, os, etc.).

2 Types de projets avec Python

Python est polyvalent et utilisé dans de nombreux domaines :

- **Développement web** : Frameworks comme Django, Flask, FastAPI pour sites, APIs, e-commerce.
- **Data Science et IA** : Bibliothèques comme Pandas, NumPy, Scikit-learn, TensorFlow pour analyse de données, machine learning, deep learning.
- **Automatisation et scripts** : Scripts pour tâches répétitives (gestion de fichiers, scraping web avec BeautifulSoup/Selenium).
- **Développement desktop** : Applications GUI avec Tkinter, PyQt, Kivy.
- **Jeux** : Pygame pour jeux 2D simples.
- **DevOps et infrastructure** : Ansible, Fabric pour automatisation serveur ; AWS/GCP via Boto3/google-cloud.
- **Sciences et recherche** : Simulations physiques (SciPy), bio-informatique (BioPython).
- **Autres** : Bots Telegram/Discord, traitement d’images (Pillow), blockchain (Web3.py).

Exemples célèbres : Instagram (Django), Netflix (data), Spotify (recommandations).

3 Installation de Python

Téléchargez Python depuis python.org (version 3.x recommandée, ex. 3.12).

Sur Windows

1. Téléchargez l'installateur .exe.
2. Exécutez-le, cochez "Add Python to PATH".
3. Vérifiez : Ouvrez cmd et tapez `python --version`.
Attendu : `Python 3.12.x`.
4. Installez pip (inclus par défaut).

Sur Linux (Ubuntu/Debian)

1. Ouvrez un terminal.
2. Installez : `sudo apt update && sudo apt install python3 python3-pip`.
3. Vérifiez : `python3 --version`.
4. Alias optionnel : `alias python=python3`.

Sur macOS

1. Téléchargez l'installateur .pkg ou utilisez Homebrew : `brew install python`.
2. Vérifiez : `python3 --version`.
3. Si plusieurs versions, utilisez `pyenv` pour gérer.

Vérification commune :

```
python --version # ou python3 --version
pip --version
```

Si erreur, vérifiez PATH ou réinstallez.

4 Premier script Python

Créez un fichier `hello.py` :

```
# hello.py
print("Bonjour, monde !")
nom = input("Entrez votre nom : ")
print(f"Bienvenue, {nom} !")
```

Exécution :

```
python hello.py
```

5 Structuration d'un projet Python

Structure recommandée pour un projet professionnel :

```
mon_projet/  
  src/  
    mon_package/  
      __init__.py  
      module1.py  
      module2.py  
  tests/  
    test_module1.py  
  README.md  
  requirements.txt  
  .gitignore  
  pyproject.toml # Optionnel pour packaging
```

- **src/** : Code source principal.
- **tests/** : Tests unitaires (avec pytest).
- **README.md** : Description, installation, usage.
- **.gitignore** : Ignorer fichiers inutiles (ex. **pycache**, **.venv**).
- **pyproject.toml** : Configuration moderne (dépendances, build).

Exemple basique pour un script simple :

```
mon_script/  
  main.py  
  requirements.txt
```

6 Gestion des dépendances

Les dépendances sont des bibliothèques externes (ex. requests, pandas).
Installez-les via pip : `pip install requests`.

requirements.txt : Liste des dépendances.

Exemple :

```
requests==2.31.0  
pandas>=1.5.0
```

Installation : `pip install -r requirements.txt`.

7 Environnements virtuels

Un virtualenv isole les dépendances par projet.

Évite les conflits de versions.

Création :

```
python -m venv .venv
```

Activation :

- Linux/macOS : `source .venv/bin/activate`
- Windows : `.venv\Scripts\activate`

Désactivation : `deactivate`.

Installez dépendances dans le venv actif.

Bonne pratique : Toujours utiliser un venv par projet.

8 Outils avancés pour l'environnement

- **pyenv** : Gérer plusieurs versions de Python.
 - **pipenv** / **poetry** : Alternatives à venv + requirements.txt, avec gestion automatique.
Exemple avec poetry : `poetry init`, `poetry add requests`.
-

9 Bonnes pratiques dès le début

- Utilisez un éditeur comme VS Code avec extension Python.
 - Respectez PEP 8 (style guide : indentation 4 espaces, noms snake_case).
 - Documentez votre code avec docstrings.
 - Testez souvent : `python -m unittest` ou `pytest`.
 - Versionnez avec Git.
-

Résumé

- Python : Interprété, dynamique, multi-paradigme.
- Projets : Web, data, scripts, etc.
- Installation : Simple sur tous OS.
- Projet : Structurez avec venv, requirements.txt.